

NL2SQL (Text-to-SQL)

NL2SQL Practice I, II

빅데이터 핀테크

Prof. Sang-Won Lee

TA. KyongShik Lee

What is NL2SQL?

- **NL2SQL** (Natural Language to SQL): 자연어 질문을 SQL로 변환하는 기술 (= Text-to-SQL)
- 자연어 질문으로 데이터 조회
- 질문의 의도와 DB 스키마에 맞는 SQL 생성

NLQ (Natural Language Question)

“급여가 2000 이상인
직원의 이름과 급여”



NL2SQL
(LLM)



SQL

```
SELECT ename, sal  
FROM emp  
WHERE sal >= 2000;
```



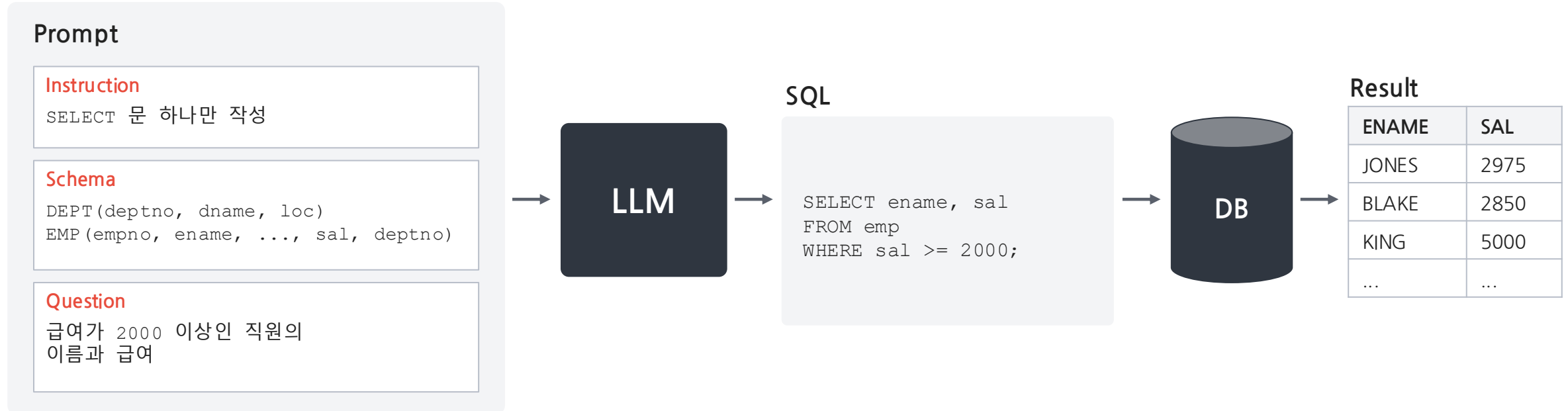
Database



Result

ENAME	SAL
JONES	2975
BLAKE	2850
KING	5000
...	...

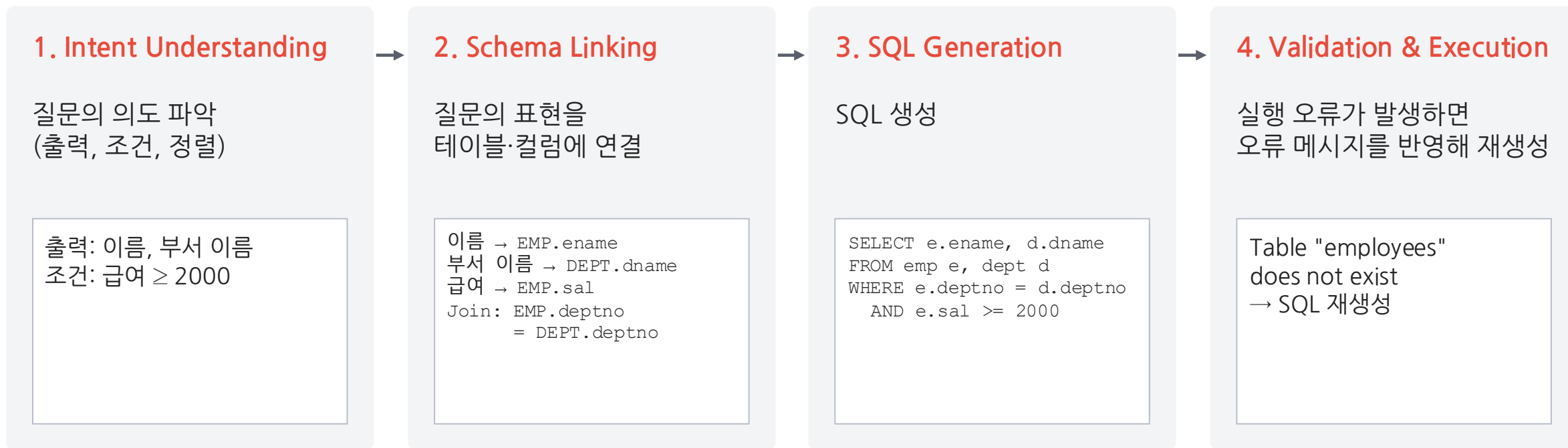
How NL2SQL Works



- LLM 입력: 지시문 + 스키마 + 질문
- 이 실습에서는 실제 DB 데이터를 프롬프트에 포함하지 않음
- LLM이 SQL을 생성하고 DB가 실행

Typical NL2SQL Pipeline

질문 “급여가 2000 이상인 직원의 이름과 부서 이름”



LLM 호출 한 번으로 1~3단계 처리. ask()에서 실행과 오류 수정 수행

Schema Linking

- 질문의 표현을 테이블·컬럼에 연결
- 스키마가 없으면 테이블·컬럼을 잘못 추측할 수 있음

질문의 표현	Schema
급여	EMP.sal
부서 이름	DEPT.dname
소속 부서	EMP.deptno = DEPT.deptno
상사	EMP.mgr = EMP.empno
급여 등급	sal BETWEEN losal AND hisal

프롬프트의 스키마 표현

Example 1. CREATE TABLE 문

```
CREATE TABLE EMP (
  EMPNO NUMBER(4) PRIMARY KEY,
  ENAME VARCHAR2(10), ...,
  DEPTNO NUMBER(2) REFERENCES DEPT);
```

Example 2. 테이블·컬럼 설명

```
EMP(empno, ename, job, mgr, hiredate,
    sal, comm, deptno)
-- 직원: 사번, 이름, ..., 급여, 커미션
-- EMP.deptno 는 DEPT.deptno 를 참조
-- EMP.mgr 는 상사의 EMP.empno
```

Challenges of NL2SQL

- 1) **Ambiguous Question** “급여가 높은 직원”의 기준 불명확: 3000 이상, 평균 이상, 상위 5명 등

- 2) **Missing Knowledge** 스키마 외 정보 필요: NULL의 업무상 의미, 고객 연봉자의 기준 등

- 3) **Complex SQL** 조인, 서브쿼리, 분석 함수가 필요한 질문

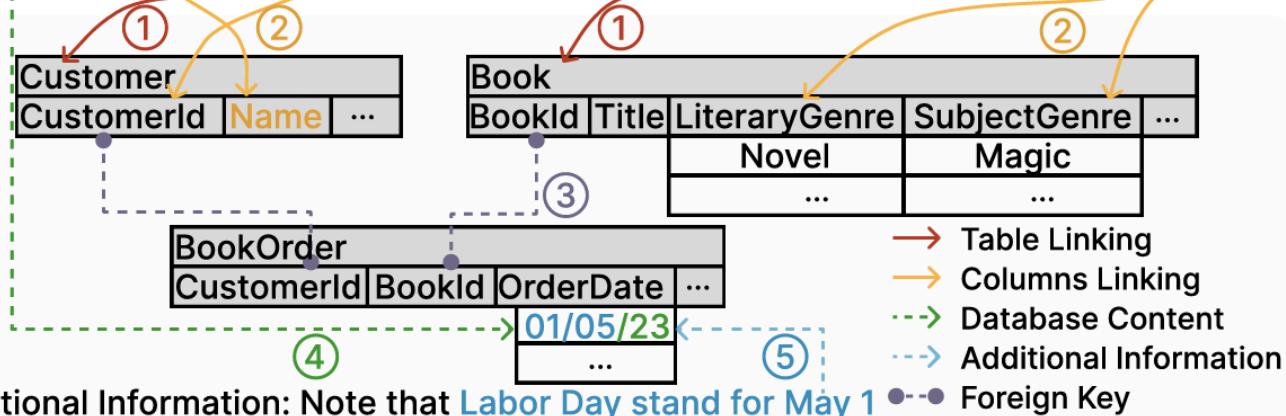
- 4) **Executable but Wrong** 실행 가능하지만 질문의 의도와 다른 SQL. 결과 검증 필요

Challenges of NL2SQL: Example

NL Query:

Find the **names** of all **customers** who checked out **books** on exactly 3 different **genres** on **Labor Day** in 2023.

Database:



Additional Information: Note that **Labor Day** stand for **May 1**

SQL:

```
SELECT Name
FROM Customer
NATURAL JOIN BookOrder
NATURAL JOIN Book
WHERE OrderDate='01/05/23'
GROUP BY CustomerId, Name
HAVING COUNT(DISTINCT SubjectGenre)=3
```

```
SELECT Name
FROM Customer
WHERE CustomerId = (SELECT CustomerId
FROM BookOrder NATURAL JOIN Book
WHERE OrderDate='01/05/23'
GROUP BY CustomerId
HAVING COUNT(DISTINCT SubjectGenre)=3)
```

1) Table Linking

customers → Customer, books → Book

2) Column Linking

genres → LiteraryGenre? SubjectGenre?

3) Foreign Key

Customer - BookOrder - Book

4) Database Content

날짜 저장 형식: '01/05/23'

5) Additional Information

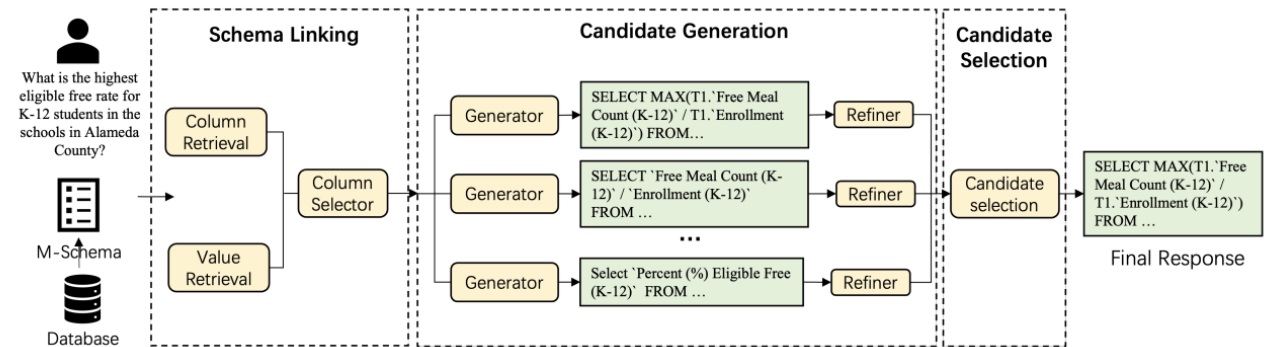
이 예제에서 Labor Day는 5월 1일

How to Improve Accuracy

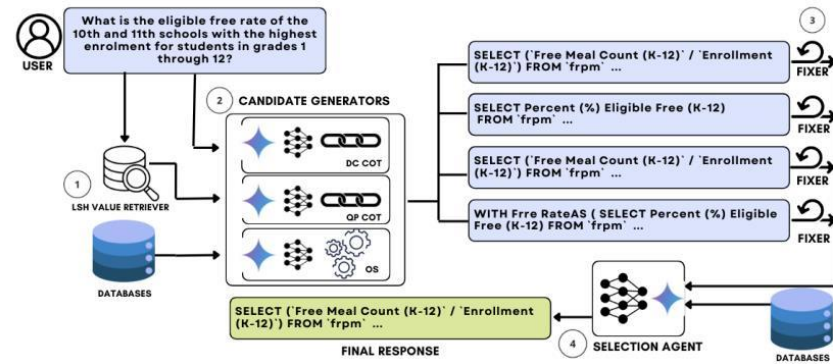
- | | |
|-----------------------|----------------------------|
| 1) Schema | 테이블·컬럼의 구조와 의미를 프롬프트에 포함 |
| 2) Hint / Dictionary | 스키마에 없는 규칙과 업무 용어의 정의 전달 |
| 3) Clear Question | 조건, 출력 컬럼, 정렬 기준을 구체적으로 작성 |
| 4) Few-shot Example | 비슷한 질문과 SQL 예시를 함께 제공 |
| 5) Execution Feedback | 실행 오류 메시지를 전달해 SQL 재생성 |
| 6) Human Verification | 생성된 SQL과 실행 결과 검증 |

Current NL2SQL Frameworks

- 스키마 선택과 SQL 후보 비교를 결합한 구조
 - Schema Linking: 관련 테이블·컬럼 선택
 - Candidate Generation: SQL 후보 여러 개 생성
 - Candidate Selection: 가장 적합한 SQL 선택



XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL



CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL

NL2SQL Platforms

- NL2SQL을 지원하는 서비스
 - <https://ai2sql.io/>
 - <https://www.text2sql.ai>
 - [SQLAI.ai: Text-to-SQL](#)
- DBMS의 자연어 질의 기능
 - [Oracle: Natural Language to SQL Generation on Autonomous Database](#)
- Snowflake: 업무 개념과 스키마를 연결하는 Semantic Layer
 - <https://docs.snowflake.com/en/user-guide/views-semantic/overview>

Hands-On

- SCOTT 데이터베이스를 이용한 자연어 질의
- 질문에 맞는 SQL 생성 및 실행 결과 비교
- 실습 데이터
 - EMP: 직원
 - DEPT: 부서
 - EMP와 DEPT의 부서번호로 연결

Practice I

- 1) Basic NL2SQL
- 2) Join
- 3) Subquery
- 4) Executable $\neq$ Correct

Practice II

- 1) GROUP BY and NULL
- 2) ROLLUP and CUBE
- 3) Analytic Function

ask() Function

ask(): SQL 생성, 실행, 결과 비교

```
ask("질문")
```

질문을 SQL로 변환해 실행하고 결과 출력

```
ask("질문", show_prompt=True)
```

LLM에 전달한 프롬프트 출력

```
ask("질문", schema="")
```

스키마를 제외한 질문

```
ask("질문", hint="...")
```

스키마에 없는 규칙과 용어를 힌트로 전달

```
ask("질문", gold=정답SQL)
```

생성된 SQL과 기준 SQL의 실행 결과 비교

실행 오류가 발생하면 오류 메시지를 반영해 SQL을 한 번 다시 생성

Practice: Join

1) Equi-Join

“각 직원의 이름과
근무하는 부서의 위치”

```
SELECT ename, loc  
FROM emp, dept  
WHERE emp.deptno = dept.deptno
```

2) Self-Join

“상사가 있는 직원의 이름과
직속 상사의 이름”

```
SELECT e1.ename, e2.ename  
FROM emp e1, emp e2  
WHERE e1.mgr = e2.empno
```

3) Join, filter

“DALLAS에서 근무하는
직원의 이름과 부서 이름”

```
SELECT e.ename, d.dname  
FROM emp e, dept d  
WHERE e.deptno = d.deptno  
AND d.loc = 'DALLAS'
```

기준 SQL과 LLM이 생성한 SQL의 결과 비교

Practice: Subquery

Subquery 1

“사번이 7566인 직원보다
급여가 높은 직원”

```
WHERE sal > (SELECT sal FROM emp
              WHERE empno = 7566)
```

Subquery 2

“CLERK이 아닌 직원 중, 적어도
한 명의 CLERK보다 급여가 낮은
직원”

```
WHERE sal < ANY (SELECT sal FROM emp
                  WHERE job = 'CLERK')
AND job <> 'CLERK'
```

Subquery 3

“직원이 한 명 이상 있는
부서의 이름”

```
WHERE EXISTS (SELECT * FROM emp e
               WHERE d.deptno = e.deptno)
```

같은 질문을 서브쿼리나 조인으로 표현할 수 있는지 비교

Executable ≠ Correct: Join and Duplicates

질문 “직원이 한 명 이상 있는 부서의 부서번호”

SQL A: Join → 14 rows

```
SELECT d.deptno  
FROM dept d, emp e  
WHERE d.deptno = e.deptno;
```

DEPTNO
20
30
30
20
...

SQL B: Join + DISTINCT → 3 rows

```
SELECT DISTINCT d.deptno  
FROM dept d, emp e  
WHERE d.deptno = e.deptno;
```

DEPTNO
10
20
30

- 직원별로 한 행을 반환하므로 같은 부서번호가 반복됨
- 중복 없는 부서 목록은 3행. DISTINCT, EXISTS, IN 등으로 작성
- 두 SQL의 중복 처리와 실행 결과를 함께 비교

Summary: Practice I

- 이 실습에서 LLM은 스키마와 질문으로 SQL을 생성하고 DB가 실행
- 스키마가 없으면 테이블·컬럼 이름을 잘못 추측할 수 있음
- 실행 여부와 별개로 SQL과 결과가 질문에 맞는지 검증

Practice II: Overview

- 1) **GROUP BY and NULL** 그룹별 집계, 그룹 조건, NULL이 포함된 산술 연산

- 2) **ROLLUP and CUBE** 소계와 전체 합계

- 3) **Analytic Function** OVER, PARTITION BY, RANK, ROWS BETWEEN

Practice: GROUP BY and NULL

1) GROUP BY + HAVING

“직원이 3명보다 많은 부서의
부서번호와 최고 급여”

```
SELECT deptno, MAX(sal)
FROM emp
GROUP BY deptno
HAVING COUNT(*) > 3
```

2) NULL in Expression

“커미션을 포함한 총 급여가
가장 높은 5명”

```
SELECT ename,
       sal + COALESCE(comm, 0)
FROM emp ...
```

- 그룹별 집계 결과의 조건은 HAVING에 지정
- sal + comm에서 comm이 NULL이면 결과도 NULL

Executable ≠ Correct: NULL in Expression

질문 “커미션을 포함한 총 급여가 가장 높은 5명”

SQL A: $sal + comm$

```
SELECT ename,
       sal + comm AS total_pay
FROM emp
ORDER BY total_pay DESC
LIMIT 5;
```

ENAME	TOTAL_PAY
MARTIN	2650
ALLEN	1900
WARD	1750
TURNER	1500
JONES	NULL

SQL B: $sal + COALESCE(comm, 0)$

```
SELECT ename,
       sal + COALESCE(comm, 0)
       AS total_pay
FROM emp
ORDER BY total_pay DESC
LIMIT 5;
```

ENAME	TOTAL_PAY
KING	5000
SCOTT	3000
FORD	3000
JONES	2975
BLAKE	2850

- $comm$ 이 NULL인 KING은 $sal + comm$ 도 NULL이므로 이 결과에서 제외됨
- NULL 처리 여부에 따른 SQL과 결과 차이 비교

Practice: ROLLUP and CUBE

1) ROLLUP 15 rows

그룹별 합계 + 직무별 소계 + 전체 합계

```
GROUP BY ROLLUP(job, deptno)
```

2) CUBE 18 rows

ROLLUP 결과 + 부서별 소계

```
GROUP BY CUBE(job, deptno)
```

ROLLUP(job, deptno) 결과 (일부)

JOB	DEPTNO	SUM(SAL)	
CLERK	10	1300	
CLERK	20	1900	
CLERK	30	950	
CLERK	NULL	4150	직무별 소계
MANAGER	10	2450	
...	...	...	
NULL	NULL	29025	전체 합계

질문: “직무와 부서번호별 급여 합계, 직무별 소계, 전체 합계”

Practice: Analytic Function

1) Aggregate vs Analytic

“모든 직원의 이름, 급여와 소속 부서의 직원 수”

```
COUNT(*) OVER (PARTITION BY deptno)
```

2) Ranking

“부서 내 급여 순위 (RANK, DENSE_RANK)”

```
RANK() OVER (PARTITION BY deptno  
              ORDER BY sal DESC)
```

3) Top-N per Group

“부서별로 급여가 가장 높은 직원”

```
SELECT ... FROM (SELECT ...,  
                  RANK() OVER (...) AS rk FROM emp)  
WHERE rk = 1
```

4) Running Total

“입사일 순서로 급여의 누적 합”

```
SUM(sal) OVER (ORDER BY hiredate  
               ROWS BETWEEN UNBOUNDED PRECEDING  
                       AND CURRENT ROW)
```

Aggregate vs Analytic Function

Aggregate Function + GROUP BY

```
SELECT deptno, COUNT(*), SUM(sal)
FROM emp
GROUP BY deptno;
```

DEPTNO	COUNT(*)	SUM(SAL)
10	3	8750
20	5	10875
30	6	9400

Analytic Function: OVER (PARTITION BY ...)

```
SELECT ename, deptno, sal,
       COUNT(*) OVER (PARTITION BY deptno) AS dept_cnt,
       SUM(sal) OVER (PARTITION BY deptno) AS dept_sum
FROM emp;
```

ENAME	DEPTNO	SAL	DEPT_CNT	DEPT_SUM
CLARK	10	2450	3	8750
KING	10	5000	3	8750
MILLER	10	1300	3	8750
SMITH	20	800	5	10875
...	...	...	...	...

- **Aggregate:** 그룹별로 한 행 반환 (3 rows)
- **Analytic:** 모든 행을 유지하고 집계 값을 추가 (14 rows)

Ranking and Top-N per Group

1) RANK vs DENSE_RANK (deptno = 20)

ENAME	SAL	RANK	DENSE_RANK
SCOTT	3000	1	1
FORD	3000	1	1
JONES	2975	3	2
ADAMS	1100	4	3
SMITH	800	5	4

- 급여가 같으면 같은 순위
- RANK: 다음 순위를 건너뛴 (1, 1, 3)
- DENSE_RANK: 건너뛰지 않음 (1, 1, 2)

2) Top-N per Group: 부서별 급여 1위

```
SELECT deptno, ename, sal
FROM (
  SELECT deptno, ename, sal,
    RANK() OVER (PARTITION BY deptno
                  ORDER BY sal DESC) AS rk
  FROM emp
)
WHERE rk = 1;
```

- 부서별 최고 급여를 받는 직원까지 식별해야 함
- 윈도우 함수는 WHERE 절에 직접 사용 불가
 - 서브쿼리에서 순위를 계산한 뒤 해당 행 선택

Running Total: RANGE vs ROWS

질문 “입사일 순서로 급여의 누적 합” (JAMES와 FORD는 입사일이 같음)

SQL A: ORDER BY만 지정

```
SUM(sal) OVER (
  ORDER BY hiredate
) AS running_sal
```

ENAME	SAL	RUNNING
KING	5000	19675
JAMES	950	23625
FORD	3000	23625
MILLER	1300	24925

SQL B: ROWS 지정

```
SUM(sal) OVER (
  ORDER BY hiredate, empno
  ROWS BETWEEN UNBOUNDED
    PRECEDING AND CURRENT ROW
) AS running_sal
```

ENAME	SAL	RUNNING
KING	5000	19675
JAMES	950	20625
FORD	3000	23625
MILLER	1300	24925

- 이 예제의 기본 범위: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
- **RANGE**: 입사일이 같은 행을 함께 합산해 JAMES와 FORD의 값이 같음
- **ROWS**: 한 행씩 누적

Optional: NOT IN vs NOT EXISTS

질문 “부하 직원이 없는 직원”

SQL A: NOT IN → 0 rows

```
SELECT ename
FROM emp
WHERE empno NOT IN
      (SELECT mgr FROM emp);
```

SQL B: NOT EXISTS → 8 rows

```
SELECT e.ename
FROM emp e
WHERE NOT EXISTS
      (SELECT * FROM emp s WHERE s.mgr = e.empno);
```

- 서브쿼리에 NULL이 포함되어 NOT IN 조건이 참인 행이 없음(0행)
- NOT EXISTS는 empno와 일치하는 mgr이 있는지 확인
- 0행 반환도 정상 실행 결과이므로 NULL 처리 방식 확인

Summary

- 중복 행, NULL, 동점, 모호한 조건에 따라 결과가 달라짐
- 생성된 SQL과 결과가 질문의 의도에 맞는지 검증
 - WHERE vs HAVING
 - GROUP BY vs OVER
 - RANK vs DENSE_RANK
 - RANGE vs ROWS